



GIỚI THIỆU GIT & GITHUB

Hệ thống quản lý phiên bản cho lập trình viên

Dựa trên The Odin Project — Foundations · **90 phút** (60' lý thuyết + 30' demo)

Mục tiêu bài học

Sau buổi học, người học có thể...



Giải thích Git & GitHub là gì

Và sự khác nhau giữa hai khái niệm này.



Phân biệt commit với Save thông thường

Cách Git ghi lịch sử thay đổi thay vì ghi đè một tệp.



Trình bày lợi ích của Git

Cho lập trình viên cá nhân và cho nhóm phát triển.



Thực hiện quy trình làm việc cơ bản

add → commit → push với các lệnh Git thiết yếu.

Git là gì?

Version Control System — hệ thống quản lý phiên bản

Hãy hình dung Git như một **“nút Save siêu cấp”** cho toàn bộ tệp và thư mục của dự án.



Save trong trình soạn thảo

Ghi đè toàn bộ nội dung vào một tệp duy nhất. Muốn giữ nhiều phiên bản phải tự nhân bản tệp:

essay-draft1.doc

essay-draft2.doc

essay-final.doc

essay-final-THẬT.doc



“Save” trong Git = commit

Ghi lại sự thay đổi của tệp/thư mục vào lịch sử đầy đủ của từng lần lưu.

→ Xem lại quá trình phát triển

→ So sánh giữa các phiên bản

→ Khôi phục bất kỳ trạng thái nào

Ba mô hình quản lý phiên bản

Pro Git, chương 1.1



Cục bộ (Local)

Lưu các phiên bản trên chính máy của bạn (vd. RCS).

Đơn giản nhưng không cộng tác được.



Tập trung (Centralized)

Một máy chủ duy nhất chứa toàn bộ lịch sử (SVN, CVS).

Máy chủ sập là tất cả tê liệt.



Phân tán (Distributed)

Mỗi máy có bản sao đầy đủ của toàn bộ lịch sử. Git thuộc loại này.

Linus Torvalds tạo ra năm 2005 cho nhân Linux.

Git và GitHub khác nhau thế nào?



Git

- ❖ Phần mềm chạy trên máy cục bộ (local)
- ❖ Quản lý phiên bản, lịch sử commit
- ❖ Hoạt động không cần Internet
- ❖ Mã nguồn mở, miễn phí



GitHub

- ❖ Dịch vụ lưu trữ từ xa (remote) trên web
- ❖ Nơi chia sẻ, cộng tác, showcase dự án
- ❖ Cần Internet để push / pull
- ❖ Một dịch vụ (còn có GitLab, Bitbucket...)

Ghi nhớ: Git làm việc cục bộ trên máy bạn; GitHub là “kho chứa từ xa” — nơi xây portfolio, kỹ năng mà hầu hết công ty phần mềm coi là bắt buộc.

Vì sao Git hữu ích?



Lập trình viên cá nhân



Xem lại toàn bộ quá trình phát triển dự án theo thời gian.



Khôi phục nhanh trạng thái cũ khi code mới gây lỗi.



Thử nghiệm tính năng trên nhánh (branch) riêng, không ảnh hưởng code chính.



Nhóm phát triển



Nhiều người sửa song song — Git hợp nhất (merge) các thay đổi.



Biết chính xác ai thay đổi gì, khi nào, vì sao.



Review code qua Pull Request trước khi hợp nhất.

Ví dụ thực tế: toàn bộ giáo trình *The Odin Project* nằm trong một repository GitHub với hàng nghìn người đóng góp.

Các lệnh Git thiết yếu

git init

Khởi tạo repository mới

git status

Xem trạng thái các tệp

git commit -m "..."

Lưu snapshot kèm thông điệp

git diff

So sánh thay đổi chưa stage

git merge <nhánh>

Hợp nhất nhánh vào nhánh hiện tại

git push

Đẩy commit lên GitHub

git clone <url>

Sao chép repo từ xa về máy

git add <tệp>

Đưa thay đổi vào vùng chờ (staging)

git log

Xem lịch sử commit

git branch / switch

Tạo và chuyển nhánh

git remote add origin

Liên kết repo cục bộ với remote

git pull

Kéo thay đổi mới nhất về máy

Cấu hình lần đầu: `git config --global user.name / user.email`

Quy trình làm việc cơ bản

Vòng lặp thường ngày của lập trình viên



```
git init # 1. Khởi tạo repo
echo "# My Project" > README.md # 2. Tạo / sửa tệp
git add README.md # 3. Stage thay đổi
git commit -m "Them README" # 4. Commit
git push -u origin main # 5. Đẩy lên GitHub
```

Ba trạng thái của tệp

modified đã sửa

staged đã vào vùng chờ (git add)

committed đã lưu vào lịch sử



PHẦN THỰC HÀNH DEMO

Đăng ký GitHub và các thao tác cơ bản — 30 phút, làm theo trên máy cá nhân

Demo 1: Tạo repo · Demo 2: Vòng lặp add–commit–push · Demo 3: Giao diện GitHub

Đăng ký tài khoản GitHub

github.com → *Sign up*

1

Truy cập github.com, bấm Sign up

Nhập email sẽ gắn bó lâu dài — email này xuất hiện trong lịch sử commit.

2

Đặt mật khẩu, chọn username

Username là “thương hiệu” portfolio: ngắn gọn, chuyên nghiệp (vd. nguyenvana).

3

Xác minh captcha + mã email

Nhập mã xác nhận được gửi tới hộp thư.

4

Đăng nhập, vào Settings

Cập nhật tên hiển thị và ảnh đại diện.



Mẹo cho giảng viên

Yêu cầu học viên đăng ký trước ở nhà — trên lớp chỉ chiếu lại các bước trong 2–3 phút.

Xác thực khi push:

- ① HTTPS + Personal Access Token — đơn giản cho người mới.
- ② SSH key (ssh-keygen → Settings → SSH keys) — tiện lâu dài.

Demo 1 — Tạo repository đầu tiên

Thao tác trên giao diện web GitHub



Bấm + (góc trên phải) → New repository



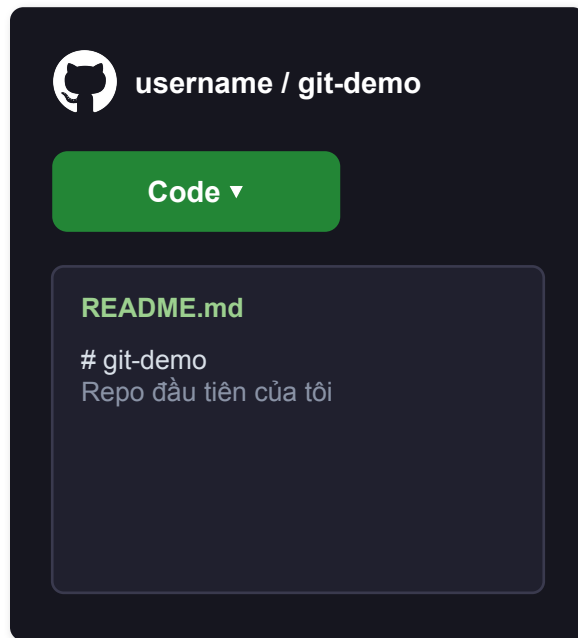
Đặt tên repo (vd. git-demo), chọn Public, tích Add a README file



Bấm Create repository



Quan sát: README.md hiển thị ở trang chủ repo; nút Code màu xanh chứa đường dẫn để clone



Demo 2 — Vòng lặp: clone → sửa → add → commit → push

```
git clone https://github.com/username/git-demo.git
cd git-demo
echo "Dong dau tien" >> README.md # sửa tệp
git status # README.md đỏ (modified)
git add README.md # status chuyển xanh
git commit -m "Cap nhat README"
git push # đẩy lên GitHub
git log --oneline # xem lịch sử
```



Điểm nhấn khi demo

Chạy git status sau MỖI bước để học viên “nhìn thấy”:

modified → **staged** → **committed**

Sau khi push, tải lại trang GitHub: README đổi nội dung, số commit tăng — khoảnh khắc “à ha” của buổi học.

Demo 3 — Thao tác trên giao diện GitHub



Sửa tệp trực tiếp trên web

README.md → biểu tượng bút chì → Commit changes. Về máy chạy git pull — minh họa hai chiều push / pull.



Xem lịch sử commit

Bấm số commit (vd. “3 commits”) → xem ai sửa gì, khi nào; mở một commit để xem diff (xanh thêm, đỏ xóa).



Star và Fork

Mở repo TheOdinProject/curriculum: Star để đánh dấu; Fork tạo bản sao về tài khoản mình — nền tảng đóng góp mã nguồn mở.



Hồ sơ cá nhân (Profile)

github.com/username: repo vừa tạo xuất hiện, ô xanh trên contribution graph — mỗi commit là một “viên gạch” cho portfolio.

Checklist giảng viên & Câu hỏi ôn tập



Trước buổi demo

- ❖ Cài Git trên máy chiếu (git --version) và đăng nhập sẵn GitHub.
- ❖ Cấu hình sẵn xác thực (token / SSH) để push không bị gián đoạn.
- ❖ Xóa repo git-demo cũ; phóng to font terminal và trình duyệt.
- ❖ Dự phòng mất mạng: ảnh chụp màn hình từng bước của Demo 1 & 3.



Knowledge check

- ❖ Git là loại chương trình gì?
- ❖ Git khác Save của trình soạn thảo thế nào?
- ❖ Git hoạt động local hay remote? Còn GitHub?
- ❖ Vì sao Git hữu ích cho cá nhân? Cho nhóm?
- ❖ Thử tự lệnh đưa thay đổi từ máy lên GitHub?

PHẦN 2: LÀM VIỆC NHÓM TRÊN GITHUB

Organization · Teams · Phân quyền · GitHub Flow

Vì sao nhóm cần **quy trình chung**?

Git cá nhân rất ổn, nhưng khi 3-4 người cùng làm một dự án (như hệ thống SmartExam), mọi thứ sẽ vỡ trận nếu thiếu "luật chơi".

Push đè & mất code

Hai người sửa cùng tệp, người push sau đè mất code của người trước. Một commit lỗi đẩy thẳng lên nhánh làm sẵn `main` bộ ứng dụng.

Không ai kiểm duyệt

Code đưa thẳng vào dự án, không ai kịp đọc, lỗi lọt ra ngoài. Cần có Organization (lưu trữ) + Phân quyền (quy tắc) + GitHub Flow (quy trình).

Tài khoản cá nhân vs. Organization

Tài khoản cá nhân

Chỉ 1 chủ sở hữu. Nếu người đó rời nhóm/xóa tài khoản, dự án sẽ biến mất. Phù hợp cho bài tập, portfolio.

Organization

Tài khoản "tổ chức" dùng chung, không phụ thuộc một cá nhân. Phù hợp cho đồ án lớp, dự án thực tế.

Teams

Trong Organization, gom thành viên thành các nhóm nhỏ (Ví dụ: Team Frontend). Giúp phân quyền hàng loạt cực nhanh.

5 Mức phân quyền Repository

Nguyên tắc đặc quyền tối thiểu: Chỉ cấp quyền vừa đủ để làm việc.

1. Read: Xem code, clone. (Dành cho khách, người quan sát).

2. Triage: Quản lý issue/PR, KHÔNG sửa code.

3. Write: Push nhánh, mở Pull Request. *(Mức độ phổ biến nhất cho thành viên lập trình).*

4. Maintain: Duyệt & merge PR. *(Cấp cho trưởng nhóm).*

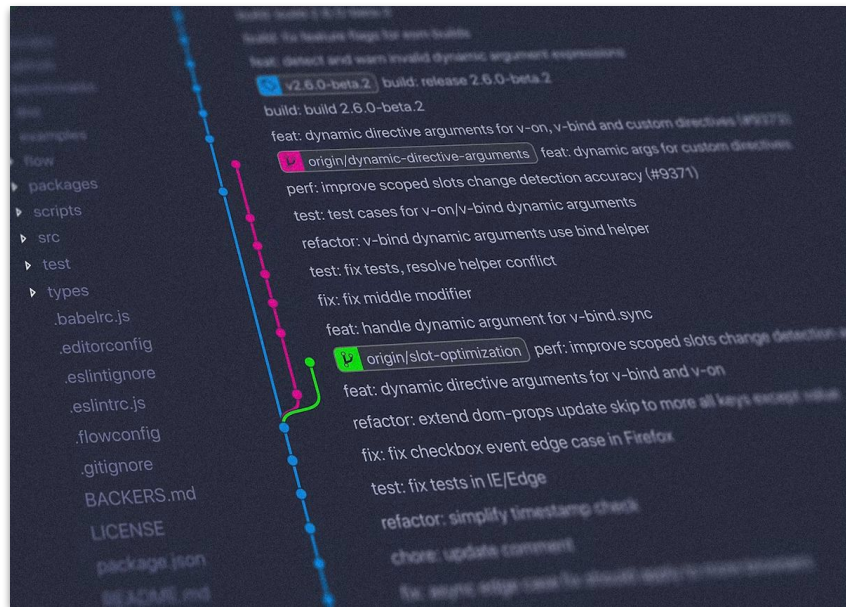
5. Admin: Toàn quyền, cấu hình repo. *(Cấp cho giảng viên/DevOps).*

Bảo vệ nhánh (Branch Protection)

Quy tắc sống còn: Cấm push thẳng vào nhánh `main` .

Phải thiết lập luật bảo vệ:

- ❖ **Require a pull request:** Mọi thay đổi code đều phải đi đường vòng qua Pull Request.
- ❖ **Require approvals:** Phải có ít nhất 1-2 người đọc và bấm "Duyệt" (Approve) mới được đưa code vào nhánh chính.
- ❖ **CODEOWNERS:** Tự động réo tên người duyệt (Cứ sửa file Database là GitHub tự gọi Team Backend).



GitHub Flow

Quy trình 6 bước **làm việc nhóm** an toàn và liên tục



Pull Request & Review Code

Pull Request (PR) là nơi "chào hàng" code. Nguyên tắc vàng: **Không ai được tự merge PR của chính mình!**

- ❖ **Tiêu đề rõ ràng:** Viết "Thêm form đăng nhập", không viết "Update".
- ❖ **Nhỏ & Tập trung:** Một PR chỉ làm một việc duy nhất để dễ review.
- ❖ **Review:** Mở tab *Files changed*, dòng màu Xanh là thêm mới, Đỏ là bị xóa. Bấm thẳng vào dòng code để để lại bình luận.



Xử lý Xung đột (Merge Conflict)

Xảy ra khi 2 người cùng sửa 1 dòng code ở 2 nhánh khác nhau. Git không tự quyết định nên sẽ hỏi con người. Đây là chuyện bình thường, không phải lỗi hệ thống!

Cách giải quyết

1. Git sẽ chèn các dấu `<<<<<< HEAD` và `=====` vào file.
2. Mở file đó ra, ngồi lại cùng nhau chọn giữ phần code nào.
3. Xóa các dấu của Git đi.
4. Chạy lại `git add` và `git commit` là xong.

Minh họa Code

```
<<<<<< HEAD
    Code của bạn ở nhánh hiện tại
=====
    Code từ nhánh main tải về
>>>>>> main
```

PHẦN THỰC HÀNH DEMO LÀM VIỆC NHÓM

Đóng vai 2 lập trình viên cùng thao tác — 30 phút

- ✓ **Demo 4:** Tạo Organization, Team và cấu hình Branch Protection.
- ✓ **Demo 5:** Thực hành trọn vẹn 1 vòng GitHub Flow (Tạo nhánh -> PR -> Review chéo).
- ✓ **Demo 6:** Cố tình tạo Merge Conflict và cùng nhau giải quyết.



Tài liệu tham khảo

- **The Odin Project — Introduction to Git**
theodinproject.com/lessons/foundations-introduction-to-git
- **Pro Git, chương 1.1–1.4**
git-scm.com/book/en/v2/Getting-Started-About-Version-Control
- **Video “What is Git?” (2 phút)**
youtube.com/watch?v=2ReR1YJrNOM
- **About GitHub and Git**
docs.github.com/en/get-started/start-your-journey/about-github-and-git
- **Repo giáo trình The Odin Project**
github.com/TheOdinProject/curriculum

Chúc buổi học vui — commit đầu tiên hôm nay là viên gạch đầu tiên của portfolio!